Prague University of Economics and Business
Faculty of Informatics and Statistics

Utilizing Vector Databases for Similarity Search

BACHELOR THESIS

Study program: Data Analytics

Author: Kateřina Bártová

Supervisor: Mgr. Martin Víta Ph.D.

Prague, May 2025

Acknowledgements

I want to thank my thesis supervisor, Mgr. Martin Víta, Ph.D., for providing the necessary data and pointing me in the right direction while writing this thesis. I want to thank my friends and family for supporting me emotionally and materially during my studies. My studies and this thesis were possible only with the help of my dear friend Vojtěch, whom I owe many thanks.

Abstrakt

Tato bakalářská práce zkoumá aplikace vektorových databázových systémů ChromaDB a QDrant, se zaměřením na jejich srovnání z pohledu uživatele a také na účinnost konkrétních textových reprezentací a metrik vzdálenosti při vyhledávání podobností. Pro provedení srovnání jsou použity tři datové sady, jedna z oblasti letectví a dvě z výzkumných a vývojových projektů. Počáteční fáze zahrnuje seznámení se s reprezentací TF-IDF, modely MiniLM-L6 a SPECTER. Jsou představeny běžné metriky podobnosti a také metriky hodnocení rank-biased overlap a precision at k. Datové sady jsou předzpracovány a poté použity k naplnění databází. Evaluace podobnostního vyhledávání se provádí s ohledem na dokumenty seřazené člověkem na základě jejich podobnosti s dotazem. Na základě experimentů s různými reprezenracemi a metrikami vzdálenosti vychází jako nejlepší reprezentace ty, vytvořené modelem MiniLm-L6, při změně vzdálenostní metriky se výsledky příliš neliší.

Klíčová slova

vektorová databáze, podobnostní vyhledávání, ChromaDB, Qdrant, slovní vnoření

Abstract

This bachelor's thesis explores applications of two vector database systems, ChromaDB and Qdrant, with focus on their comparison from a user's perspective and the effectiveness of specific text representations and distance metrics in similarity search. In order to perform the comparison, three datasets are utilised, one from the aviation domain and two from research and development projects. The initial phase involves familiarising with TF-IDF representation, MiniLM-L6 and SPECTER models. Standard similarity metrics and evaluation metrics, rank-biased overlap, and precision at k, are introduced. The datasets are pre-processed and then used to fill the databases. The similarity search evaluation is done concerning human-sorted documents based on their similarity to the query. Based on experiments with various representations and distance metrics, those created by the MiniLM-L6 model emerge as the best representation; when changing the distance metric, the results do not differ significantly.

Keywords

vector database, similarity search, ChromaDB, Qdrant, embedding

Contents

Introduction

This bachelor's thesis concerns vector databases, their usage in similarity search, and the effectiveness of specific text representations.

One of the goals of this thesis is to try utilising a vector database for semantic search using free Python libraries. Databases of choice are Qdrant (Qdrant, 2024a) and Chroma (Chroma, 2024).

While implementing the search, the question of representation arises. The next goal of this thesis is to briefly look into how much the choice of used embeddings and vector representation can influence the effectiveness and performance of the search itself, as well as the database's storage time and space usage. In this work, we are comparing simple TF-IDF vectors, pre-trained embeddings using the fast model *"all-MiniLM-L6-v2"*, and a model trained on scientific papers, *"SPECTER"*, both from the Python transformers library. The central hypothesis is that sentence embeddings are much more accurate when retrieving documents.

Creating a database of documents with a similarity search can also be beneficial for businesses and their company documentation, where users can find documents more effectively, as they are not searching for their name but for their contents. With this in mind, we are working with two records datasets in this thesis. *Aerospace* contains information about companies from the aerospace industry, to be able to find the best suitable company to provide aeroplane parts. *Projects* is a collection of Scientific projects and their goals, which could be used to look for similar projects and their contributions when starting a new one. Similar usage can be inferred from the last dataset - Articles - which contains the titles and abstracts of articles concerning computer science.

Aerospace and Projects datasets are used to familiarise with and test the databases, and the Articles dataset is used to test the chosen text representations.

The ranking results of similarity search are evaluated using human-sorted referential ranks and metrics such as rank-biased overlap (RBO) and precision at k (P@k).

ChromaDB and Qdrant databases are then compared from the user's perspective with criteria such as speed of inserting data, use intuitiveness and customisability.

1. Vector database

Databases, in general, are a way of storing various kinds of data and are used to search them for relevant information. In the context of this work, vectors are meant as numeric representations of text, and the way they are created is described in the next chapter.

The main difference between relational and vector databases is their implementation and usage. Although queries in relational databases use precise criteria, typically described in a query language like SQL, and their evaluation is based on whether they are met, vector queries are assessed by measuring similarity or distance (Pan et al., 2023). This can also cause problems with interpreting the results, as the "closest" result does not have to be the best fit. As there are often neural networks involved in the creation of vectors, the result's proximity to the query can be impossible to explain. Another problem is that, as there can be a "closest match" present, it may not be what the user is looking for if the given document is missing from the database.

The market for vector databases is quickly growing (The Business Research Company, 2025); with well-known names such as MongoDB and Pinecone, we chose two open-source platforms with Python implementations, Chroma and Qdrant. They are both advertised as easy to use and beginner-friendly, and according to their websites, offer free implementations with possible cloud usage.

1.1 ChromaDB

The first database of choice is a project from San Francisco, USA. It is available in Python as a library *chromadb* (Huber, 2024), which allows users to work with its API and a local client. Its everyday use case is for LLM applications and RAG (retrieval augmented generation)(MSV, 2023). The version used for experiments in this thesis is chroma 0.5.13.

1.2 Qdrant

Qdrant is a new (started in 2021) production-ready database with high customizability. It also offers cloud services for startups and big companies. Its everyday use is in recommendations and RAG applications (Qdrant, 2025). Qdrant is available as a Python library *qdrant_client* (Vasnetsov, 2024) and can run as a local client. The version used in this thesis is qdrant-client 1.12.1

2. Text Representations

As humans, we often give meanings to words; however, we cannot comprehend their semantics if those words are in a language we do not know. If we provide a text and a question to a machine, it can only look for similarities in common character sequences. On the other hand, when we change text into numbers, we give it another dimension by encoding the meaning within the numerical representation. We then move abstract connections to concrete values with measurable distances and possible operations. In this work, we are interested in a few text representations based on the input text and text representations based on a pre-trained encoder-decoder architecture (Yang et al., 2024). In this way, we transform text and its meaning into numerical vectors that can convey this meaning and relations, where semantically close texts have smaller distances, e.g. cosine distance, in the vector space.

Since we need to describe the whole document as a one-dimensional vector, we utilise the Term-Frequency - Inverse Document Frequency (TF-IDF) as a baseline model, followed by sentence and document embedding models.

2.1 TF-IDF

When we want to work with only the provided data and not pre-learned or pre-labelled texts, a great option is *term frequency – inverse document frequency*, which derives the importance of words based on their occurrences in a given set of texts.

In the scikit-learn library's (Pedregosa et al., 2011) *TFIDFVectorizer*, TF-IDF is defined using term frequency $TF(t, d)$, the number of times a term t occurs in a document d and inverse document frequency $IDF(t)$, which is computed as:

$$IDF(t) = log\left(\frac{1+n}{1+DF(t)}\right) + 1$$

, where n is the total number of documents in the dataset and $DF(t)$ document frequency, i.e. the number of documents in which term t is present. The $TF-IDF(t, d)$ is then defined as:

$$TF - IDF(t, d) = TF(t) \times IDF(t)$$

(scikit-learn developers, 2025).

This definition differs from the standard examples, it is however worth mentioning as the experiments are utilizing this implementation.

In textual data, we are not interested in terms that occur frequently but bear little to no meaning. Such words are called *stopwords* and are generally removed as part of the corpus preprocessing. In this work, we experiment with the efficiency of TF-IDF representation with and without this preprocessing step.

There could be many reasons for using this method of text encoding. The vectors are theoretically explainable, and the process is simple enough to be done by hand. On minor texts, it can be fast; however, with the growing length of documents and their number, the vector representing each document grows as well and is often sparse since the indices of a vector have to be able to capture all words used.

In this work, TF-IDF vectors are used as baseline examples of storing textual data and their meaning to be used for similarity search.

2.2 MiniLM-L6

MiniLM models are sentence and short paragraph encoders.(Wang et al., 2020) MiniLM-L6 maps the input text into a 384-dimensional dense vector space. (Reimers & Gurevych, 2019)

This obtained sentence embedding is a dense vector representation that captures its semantic meaning. At its core is the capacity to predict the surrounding sentences given an input sentence. (Pilehvar & Camacho-Collados, 2021)

This model was chosen because it is set as the default embedding function in ChromaDB, which prompted testing its effectiveness.

2.3 SPECTER

SPECTER (Scientific Paper Embeddings using Citation-informed Transformers)is a pre-trained transformer-based model for representing documents.(Cohan et al., 2020) It operates on the encoder-decoder architecture, which means that it first processes the input text, creates a sequence of hidden states capturing the context and then generates an output sequence. (Yang et al., 2024)

This model was chosen because it was built on SciBERT and is able to work with domain-specific data (Cohan et al., 2020), which is helpful as the Projects and Articles datasets are focused on scientific texts.

3. Similarity search

The similarity search pipeline, in our case, consists of encoding the query text, using the same encoding method for both query and the document, choosing a distance metric, computing the distances (or similarities) of document-query pairs, and sorting documents from the most similar or least distant. In the chapter 5, we show how the used representations and metrics influenced query results.

3.1 Distance metrics

After obtaining vectors and creating a vector database, the next step is to decide how the similarity of those vectors should be measured. Since the vectors are geometric objects, we can measure their distance within the vector space defined by the vector database. Chroma and Qdrant allow us to use cosine similarity, Euclidean (L2) distance as well as dot product (inner product). These metrics are well known and in this work, are defined in the same fashion as in the book *Statistics for Machine Learning* by Pratap Dangeti and *Chroma* documentation.

The implementations of respective distance metrics vary with the use of different vector databases. In the cosine example, we show both *distance* and *similarity* as Qdrant's implementation uses similarities while Chroma works with distances. In the case of Euclidean distance, Chroma uses the squared value, whereas Qdrant uses the absolute length of the distance vector.

3.1.1 Cosine similarity

This metric measures the cosine of the angle between the vectors, so that if they are pointing in similar directions, they are considered close, no matter their length, which can sometimes bring unreliable results. It is robust and works with sparse and high-dimensional data. The cosine similarity is defined as

$$s = \frac{\sum(A_i \times B_i)}{\sqrt{\sum(A_i^2)} \cdot \sqrt{\sum(B_i^2)}},$$

where A_i and B_i are values of vectors A and B with index i. The distance metric is then

$$d = 1 - s = 1 - \frac{\sum(A_i \times B_i)}{\sqrt{\sum(A_i^2)} \cdot \sqrt{\sum(B_i^2)}}.$$

To speed up the computation during query results, Qdrant normalises the vectors when saving them. (Qdrant, 2024a)

3.1.2 Dot product

In Chroma's implementation, the dot product is called the *inner product*. The dot product is an arithmetic operation on vectors that outputs a scalar number. It is defined with A_i and B_i being i-th values of vector A and B, respectively, as

$$s = \sum(A_i \times B_i).$$

Again, the distance is computed as

$$d = 1 - s = 1 - \sum(A_i \times B_i).$$

This method is often used when the vectors are normalised, which can speed up querying because the operation is simpler and faster than cosine distance.

3.1.3 Euclidean distance

The Euclidean distance metric treats vectors as point coordinates and computes the length of the connecting vector. Similarly to other techniques, Chroma uses different score - its *squared L2* is defined as

$$d = \sum(A_i - B_i)^2,$$

whereas, in Qdrant, the implementation works with *Euclid distance*,

$$d = \sqrt{\sum(A_i - B_i)^2},$$

with A_i and B_i the values on the index i of vectors A and B. Theoretically, Chroma's squared computation should be faster because of the missing square root operation. This metric is often used because of its interpretability; however, it is not suitable for working with sparse vectors.

3.1.4 Manhattan distance

Manhattan setting in newer versions of Qdrant allows users to utilise the L1 metric defined by

$$d = \sum |A_i - B_i|,$$

again, A and B are vectors and i denotes the index of their value. This metric was mentioned only for the sake of completeness, and it will not be used in the experiments, as this setting is possible only in one of the tested platforms.

3.2 Evaluation

To assess the quality of the results, we are comparing referentially sorted documents, with the first being the most similar document to the query and documents sorted by the database. We approach this task as if we had an unreasonable number of candidates for being the relevant retrieved documents; however, we only consider the top k of the sorted results, as our theoretical user would consider $k + n$-th results as irrelevant. We describe this in an example of a Google search, which shows ten results per page, and the user loses interest after discovering the first seven.

While Kendall's tau or Spearman's footrule are broadly used, they work on complete lists in their base form, which would not yield a valuable result.

This task can be viewed from two perspectives - a ranking task or a classification task. The user can look through all the results shown and choose the best for their use, or go through the list in order and stop at some point and not look at the rest.

Here, I chose two evaluation strategies - precision, often used for evaluation of classification models (Kotu & Deshpande, 2019) and rank-biased overlap, used to compare ranked lists with weights considering the ordering. (Webber et al., 2010)

3.2.1 Rank-biased Overlap (RBO)

Proposed in 2010, RBO uses the proportion of the overlapping elements while considering the ranking in the list and adding weights accordingly. It can be used to calculate the similarity between definite and indefinite lists.

RBO is defined on infinite lists as:

$$RBO(S, T, p) = (1 - p) \sum_{d=1}^{\infty} p^{d-1} \cdot A_d,$$

where S and T are the two ranked lists, p is the users *persistence* i.e. the probability that they will look at another item in the list. d denotes the current depth and A_d is the sum of *agreements* at depth d, with agreements defined as the proportional intersection in lists

$$A_d = \frac{|S_d \cap T_d|}{d}.$$

(Webber et al., 2010)

The results are in the range [0, 1], with the lower bound describing a completely disjoint pair of lists (lists containing completely different items) and the upper bound describing an identical pair of lists with the same ordering. (Webber et al., 2010)

The interpretation is that of a probabilistic model with a user comparing the pair of lists. They look at the top of the lists and have a probability p of continuing to the next item and a probability $1 - p$ of stopping. (Webber et al., 2010)

We will use d of 7, as we presume that if our user were using Google search instead, they would stop at the seventh result and get bored or lose interest afterwards.

This computation is implemented as a Python library *rbo*. (Chen, 2023)

3.2.2 Precision at *k* (P@k)

If we assume the example user does not care about the ordering and looks at all the shown results and chooses which is the best for them, we can think of the assignment as classification. The question then would be whether the critical documents were or were not retrieved in this shown set. This can be quantified by *precision* metric, which is defined by

$$Precision = \frac{Number\ of\ relevant\ retrieved\ documents}{Number\ of\ all\ retrieved\ documents}.$$

The *relevant documents* are defined as documents that are in the same top k set of reference rankings and $Number\ of\ all\ retrieved\ documents$ are equal to k as that is the set we are considering. This metric ranges from 0 to 1, where 0 means no relevant documents were retrieved in the top k and 1 means all k documents are relevant (same as the reference set) regardless of their order.

4. Datasets

Three data sets - Aerospace, Projects, and Articles - are utilised in the experiments.

The first dataset comprises aviation industry companies, mainly their names and descriptions. This dataset is relatively small and it is used to explore the database's usage options and performance.

The second dataset, Projects, is from the domain of science, research and development projects paid for by the Czech government. The presence of typing errors and domain-specific phrases in the dataset, in combination with a larger data volume, makes this dataset a good candidate to test the performance of different vector databases.

The last dataset, Articles, contains titles, abstracts, and keywords of articles from the domain of computer science. This collection serves as the primary data source for experimenting with text representations in this work. In addition, there is also a set of three similarity search queries with human-evaluated rankings that serve as the reference rankings for the information retrieval systems.

4.1 Aerospace

The database is filled with data concerning companies whose business occurs in the aerospace industry and their descriptions. The data was scraped from a set of websites of companies of interest. The source URLs are part of the dataset.

The unpreprocessed dataset consisted of 606 rows and 4 columns with names (*Name*), web addresses (*URL*), French descriptions (*FR*) and, in 327 cases, English descriptions (*EN*). The remaining English descriptions were machine-translated. Out of 606 rows, 17 rows had to be deleted as duplicates. The duplicate check was done only on the first three columns, as English translations of the same French text differed, possibly due to machine translation tools' caveats. Furthermore, 22 more rows were omitted due to the presence of missing values, as the French and English descriptions were unavailable and could not be imputed. The last two steps were removing the French column (*FR*) due to obsolescence and stopword removal performed on the column of English texts (*EN*) for TF-IDF encoding. The stopwords list was sourced from the SpaCy Python library (AI, 2025). Furthermore, texts were also stripped of non-alphanumeric characters and put into lowercase.

The final preprocessed dataset consisted of 5 columns and 543 rows.

column name	data type	example data
id	numeric, primary key	100
name	string	AIRCRAFT-DISMANTLING
url	string	https://www.ac-dismantling.com
en	string	Removal, dismantling and upgrading of private and business aircraft
en_filtered	string	removal dismantling upgrading private business aircraft

Table 4.1: Columns of pre-processed Aerospace dataset, source: author

4.2 Projects

This 'Projects' collection contains information about selected projects from the information portal IS VaVaI (Informační systém výzkumu, vývoje a inovací), the information system of research, development and innovation with projects that have been funded by the Czech government. The data were scraped from the information system and 19,460 URLs, project codes, names of projects in English, keywords, project goals in English and identification numbers of recipients were collected. The VaVaI information system is also the source of the Articles dataset.

After conducting the analysis of the dataset, the only missing values are present in the *name* and *goal* columns, as this information cannot be reliably imputed; these rows were dropped, totalling 121 rows. Duplicate check shows identical values in *name*, *goal*, and *keywords* columns. However, this is not an undesired state, as our case projects are identified by their code and URL address, and it is entirely reasonable for projects to share goals or names, especially for more generic ones. The single duplicate row was found and deleted. As the texts were not translated during editing, but rather set by users of the information system, the next step is a language check. For this task, package *langdetect* (Nakatani, 2010) was used, and texts with the English language were detected. Texts with a confidence score for the English language lower than 0.6 were reviewed manually. The overwhelming majority of texts were indeed in English and falsely flagged, and only individual rows were incorrect, so the data was left unchanged. Lastly, stopwords were removed for TF-IDF processing, and texts were changed to lowercase; this result is in the *filtered* column.

The pre-processed dataset has 7 columns and 19,338 rows.

column name	data type	example data
id	numeric, primary key	36
url	string	https://www.isvavai.cz/cep?s=jednoduche-vyhledavani&ss=detail&h=7AMB14DE003
code	string	7AMB14DE003
name	string	Interactions effects and collective behavior in driven diffusion systems
goal	string	The aim is to design a theoretical analysis of stochastic models of particle system dynamics.
keywords	string	diffusion systems
identification	numeric	216208
filtered	string	interactions effects collective behavior driven diffusion systems aim design theoretical analysis stochastic models particle system dynamics

Table 4.2: Columns of pre-processed Projects dataset. Source: author

4.3 Articles

The final 'Articles' dataset is a set of 318 rows, each containing scientific journal articles and conference papers spanning a range of computer science themes and their applications in various domains. All of the sourced articles and papers were produced at the Institute of Computer Science of the Czech Academy of Sciences in 2023 and 2024. The following experiments will deal only with these publications' names and abstracts. The texts were downloaded on May 13th, 2025, from the website of IS VaVaI (namely https://www.isvavai.cz/riv) by this thesis' supervisor. The VaVaI information system is also the source of the Projects dataset.

The source file was inspected for missing data and duplicates. However, no additional changes were needed in this domain. In the following experiments, we focus primarily on the *abstracts* column. Its most common words are "paper", "study" and "work".

The most used bigrams and trigrams are shown in the following plot and show that the works are indeed centred around Computer science topics such as artificial intelligence, machine learning and neural networks; there are also the 95 per cent confidence intervals mentioned often, and the theme of climate change and thermal comfort is prominent.

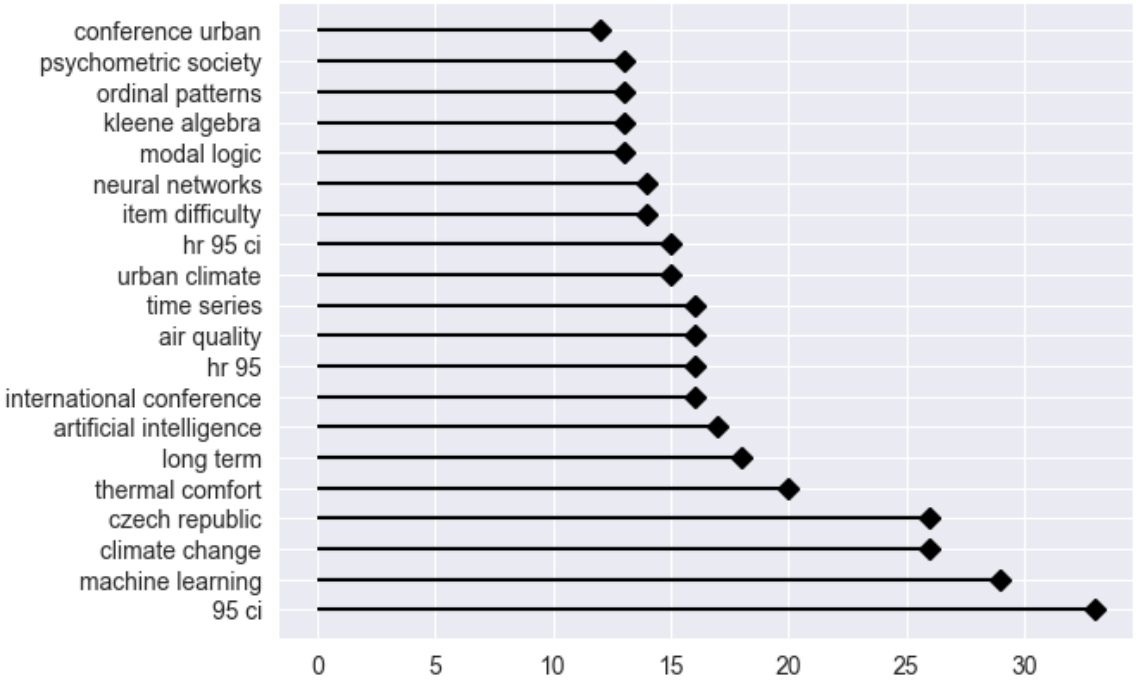

Figure 4.1: Most common bigrams and trigrams in the *abstract* column, source: author

The lengths of the abstracts range from 13 to 547 words, with a mean of 177.3. The distribution of word counts is shown below.

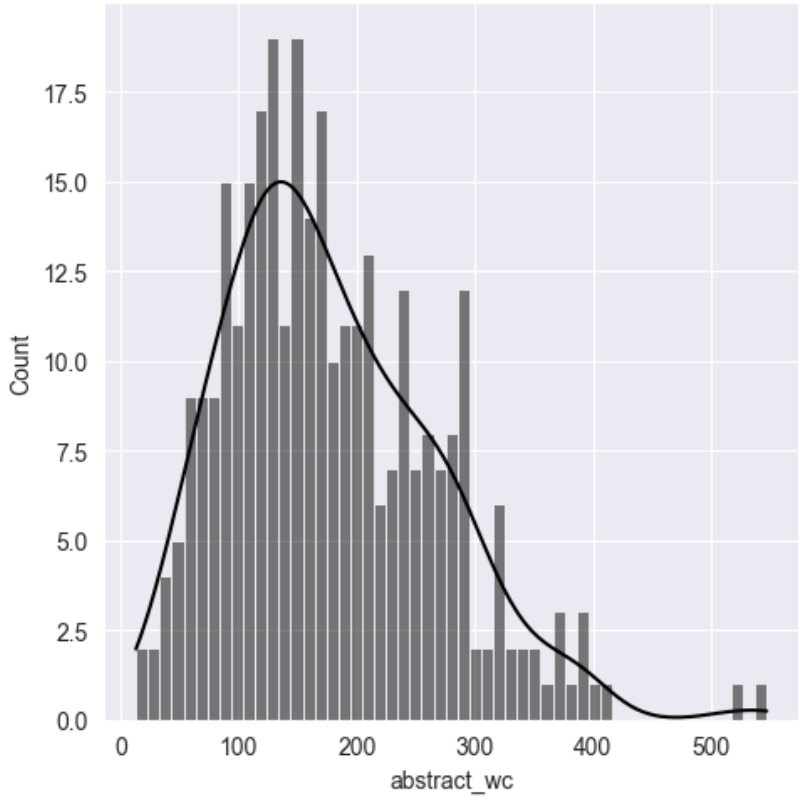

Figure 4.2: Histogram of word counts of entries in *abstract* column, source: author

Stop-word removal was performed on the *abstract* column, which resulted in creating the *abstract_clean* column for use with TF-IDF representation. The usual set of English stop-words from the Spacy package was used without any additions. *Abstract* column contains 11,472 unique terms. After cleaning, *abstract_clean* contains 7288 unique words.

The resulting csv file contains 318 rows and 5 columns.

column name	**data type**	**example data**
id	numeric, primary key	140
title	string	ShinyItemAnalysis: Test and Item Analysis via Shiny, v. 1.5.0
abstract	string	Package including functions and interactive shiny application for the psychometric analysis of educational tests, psychological assessments, health-related and other types of multi-item measurements, or ratings from multiple raters.
keywords	string	student assessment;item analysis;item response theory
abstract_clean	string	Package including functions interactive shiny application psychometric analysis educational tests psychological assessments health related types multi item measurements ratings multiple raters

Table 4.3: Columns of pre-processed Articles dataset. Source: author's rendition

4.4 Queries

The queries were hand-crafted to test the capabilities of databases and the performance of different distance metrics and embedding methods used for information retrieval. Each dataset has its own set of queries. There were two approaches to creating and using queries:

- Two queries were constructed for the Aerospace and Projects datasets each. The first query consists of one word, specifically selected to try to be as close to as many rows as possible. The second, longer, query is a *lorem ipsum* text, which should be semantically distant from all rows. The goal is to test whether the database does or does not return any results and to log the time it takes to do so.
- A total of three queries were created for the Articles dataset, each replicating one selected document from the dataset. In the experiment, we are interested in finding the top k closest documents, simulating a situation where the user is trying to find additional information for their own research. These queries each have a ground truth list of the top 7-9 ranked documents, which were sorted by a human as the closest

matches. The ranks are then compared to the results gained from the search and evaluated using measures mentioned in 3.2.

query text
aircraft
Lorem ipsum dolor sit amet, consectetur adipisici elit, sed eiusmod tempor incidunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquid ex ea commodi consequat. Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Table 4.4: Queries of the Aerospace dataset. Source: author

query text
project
Lorem ipsum dolor sit amet, consectetur adipisici elit, sed eiusmod tempor incidunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquid ex ea commodi consequat. Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Table 4.5: Queries of projects dataset

query title	**top result title**	**results**
Appropriate artificial intelligence algorithms will ultimately contribute to health equity	Ethical Aspects of Information-Based Medicine (With a Focus on Mental Health)	9
Argument Classification with BERT plus Contextual, Structural and Syntactic Features as Text	Argument Mining with Modular BERT and Transfer Learning	7
EEG Connectivity in Treatment of Major Depressive Disorder: Tackling the Conductivity Effects	Phase-based Causality Analysis of EEG in Treatment of Major Depressive Disorder	9

Table 4.6: Queries of Articles dataset. Source: author's rendition

5. Comparison of used text representations

This experiment was done solely on the Articles dataset. The chosen text representations were created, inserted into the vector databases, then queried using previously mentioned distance metrics, and the ranks of retrieved documents were saved. Both Qdrant and Chroma databases returned the same results using the same text representations, distance metrics and queries. Since the query documents are included in the corpus, the query document itself should be at the top-ranked position with 100 % similarity. This proved to be true in all experiments, so in the results, this top-ranked document is omitted, and the results show the top seven *other* similar documents.

5.1 TF-IDF

The TF-IDF representations of documents and queries for the information retrieval system were used as the baseline for this experiment. They are simpler than the other utilized methods of encoding. The abstract column of the Articles dataset contains 11,472 terms, after stop-word removal, 7,728 unique terms. There was no problem fitting these vectors into the databases, so they did not have to be shortened. The following table shows the identification numbers of each query's top 7 ranked documents using TF-IDF encoding with stopwords included.

q1 reference	**q1 COSINE**	**q1 DOT**	**q1 EUCLID**
220	**267**	**267**	**267**
81	213	213	213
100	28	28	28
60	238	238	238
267	163	163	163
118	91	91	91
248	**220**	**220**	**220**
q2 reference	**q2 COSINE**	**q2 DOT**	**q2 EUCLID**
17	**17**	**17**	**17**
161	311	311	311
60	177	177	177
297	163	163	163
157	31	31	31
253	18	18	18
80	200	200	200
q3 reference	**q3 COSINE**	**q3 DOT**	**q3 EUCLID**
115	**115**	**115**	**115**
114	**111**	**111**	**111**
111	308	308	308
295	97	97	97
31	**295**	**295**	**295**
113	**31**	**31**	**31**
269	24	24	24

Table 5.1: IDs of top 7 documents using TF-IDF, source: author

The mean RBO over all distance metrics is 0.37. All distance metrics resulted in similar rankings, with RBO of 0.09 on the first query, 0.37 on the second query and 0.64 on the third. The precision at $k = 7$ is 0.29 on the first query, 0.14 on the second query and 0.57 on the third, which means that 2, 1 and 4 documents were correctly classified as *top 7 most relevant* to each query. The mean precision at 7 is then 0.33.

The second experiment involved the use of cleaned abstract texts. The resulting ranks were slightly different from those obtained using TF-IDF terms with the stopwords included; however, all distance measures returned the same results. The following table shows those results.

q1 reference	**q1 COSINE**	**q1 DOT**	**q1 EUCLID**
220	**267**	**267**	**267**
81	213	213	213
100	**220**	**220**	**220**
60	154	154	154
267	**60**	**60**	**60**
118	140	140	140
248	318	318	318
q2 reference	**q2 COSINE**	**q2 DOT**	**q2 EUCLID**
17	**17**	**17**	**17**
161	31	31	31
60	177	177	177
297	**161**	**161**	**161**
157	18	18	18
253	**253**	**253**	**253**
80	95	95	95
q3 reference	**q3 COSINE**	**q3 DOT**	**q3 EUCLID**
115	**115**	**115**	**115**
114	308	308	308
111	**111**	**111**	**111**
295	97	97	97
31	**31**	**31**	**31**
113	295	295	295
269	24	24	24

Table 5.2: IDs of top 7 documents using TF-IDF with stopwords removed. Source: author

In the first and second query results, there are 3 documents that were correctly ranked among the top 7 most relevant; in the third query, 4 of the 7 retrieved documents are also in the reference set. That makes the average precision at 7 equal to 0.47. The average rank-biased overlap is at 0.49 with 0.30 for the first query, 0.52 for the second query and 0.64 for the third query. From these results, it would seem that in the second query, the database matched the three relevant documents closer to their reference ranks.

5.2 SPECTER

Pre-trained document-level embedding model SPECTER was chosen as it was specifically created and trained for working with scientific texts and their abstracts. Its usual usage is for title-abstract pairs; however, in this work, we are encoding only the *abstract* column for keeping simplicity and consistency with other models. In the experiment, *"allenai/spectre"* initialised model from Huggingface's Transformers library was provided to the database, and

the encoding part was done automatically in the background according to the database's implementation on insertion. The table below shows that the returned ranks differ according to the chosen distance metric.

q1 reference	**q1 COSINE**	**q1 DOT**	**q1 EUCLID**
220	**220**	**220**	**220**
81	**267**	**267**	**267**
100	**81**	214	**81**
60	214	197	214
267	**60**	100	**60**
118	197	**118**	68
248	**118**	**60**	**118**
q2 reference	**q2 COSINE**	**q2 DOT**	**q2 EUCLID**
17	**17**	**17**	**17**
161	151	151	151
60	177	177	177
297	312	312	11
157	11	**157**	312
253	**157**	11	81
80	73	73	**80**
q3 reference	**q3 COSINE**	**q3 DOT**	**q3 EUCLID**
115	**115**	**115**	**115**
114	**269**	308	**269**
111	308	**111**	308
295	199	**269**	199
31	231	199	231
113	**114**	**114**	**114**
269	**111**	231	**111**

Table 5.3: IDs of top 7 documents using SPECTER embeddings. Source: author

All distance metrics have very similar sets of top 7 documents, only differing in the relevance order. In some cases, there are distinct documents unseen in other sets. For query one, all sorted lists contain 5 relevant documents. In case of query two, the number of correctly classified as 7 most similar is 2, in case of the third query, there are 4 correctly ranked documents. The mean precision at 7 is then 0.52. The highest mean RBO is the dot product metric with 0.54, followed by cosine similarity with 0.52 and Euclidean similarity with 0.51. The dot product has successfully hit the "correct" spot on the list most times, but the differences in all results are minimal.

5.3 MiniLM-L6

MiniLM is an embedding model that was chosen due to its universality and many use cases. Chroma uses this model as a default embedding model, giving more reason to compare it with other text representation models and methods. The model was used with the Qdrant database in the same way as the SPECTER model via HuggingFace as a sentence transformer (*"sentence-transformers/all-MiniLM-L6-v2"*). Again, all distance metrics returned the same results in the top 7 places, this time with higher precision than TF-IDF.

q1 reference	**q1 COSINE**	**q1 DOT**	**q1 EUCLID**
220	**220**	**220**	**220**
81	**267**	**267**	**267**
100	**81**	**81**	**81**
60	311	311	311
267	100	100	100
118	214	214	214
248	193	193	193
q2 reference	**q2 COSINE**	**q2 DOT**	**q2 EUCLID**
17	**17**	**17**	**17**
161	**157**	**157**	**157**
60	177	177	177
297	73	73	73
157	**161**	**161**	**161**
253	**297**	**297**	**297**
80	**60**	**60**	**60**
q3 reference	**q3 COSINE**	**q3 DOT**	**q3 EUCLID**
115	**115**	**115**	**115**
114	**269**	**269**	**269**
111	**111**	**111**	**111**
295	**295**	**295**	**295**
31	199	199	199
113	308	308	308
269	**114**	**114**	**114**

Table 5.4: IDs of top 7 documents using MiniLM embeddings. Source: author

The ranks using MiniLM L6 have reached RBO of 0.67 on the first query results, 0.58 for the second query and 0.68 on the third. These results as a whole are the best of all tested configurations. Only in case of query one, SPECTER with cosine and Euclidean distance was better at 0.69; however, the mean RBO over all queries is unmatched. Also in precision measure, MiniLM excels with the average precision at 7 over all queries of 0.67. Query one returned 4 correct documents, query two and three returned 5 relevant documents each,

which means that the database with texts encoded using the Minilm model included only 7 documents, which a human would not sort into the top 7 most similar in relation to all three queries.

5.4 All Representations

Using rank-biased overlap, the representations and distance metrics can be compared next to each other. From the following plots, we can see that MiniLM-L6 outperforms all other models in most cases. There is also a visible difference between TF-IDF encodings with and without stopwords, where the cleaned texts and therefore denser vectors are able to capture more of the abstract's meaning, and the retrieval works better. In one Instance, SPECTER outputs a ranking with a higher RBO. However, that difference is minimal, and it can be attributed to only sorting.

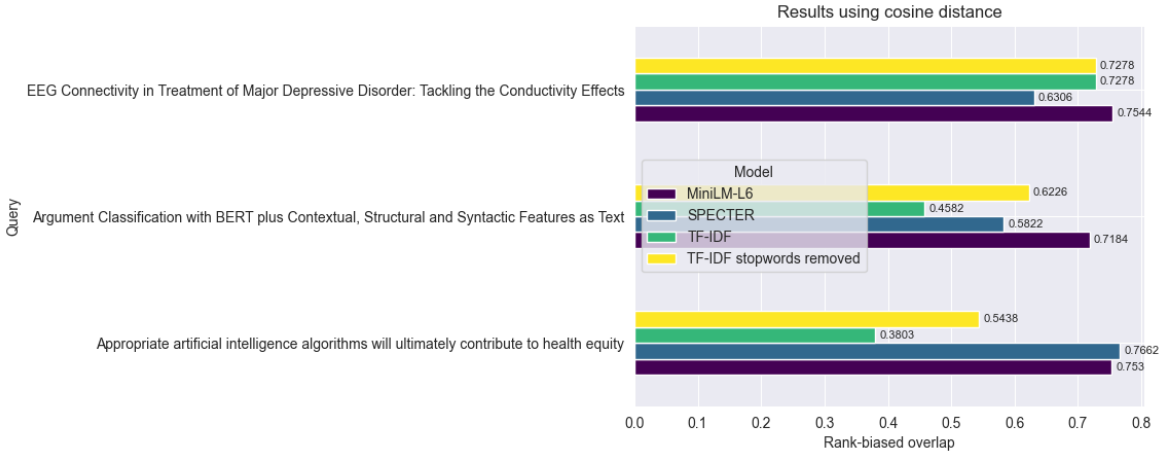

Figure 5.1: Results using cosine distance, source: author

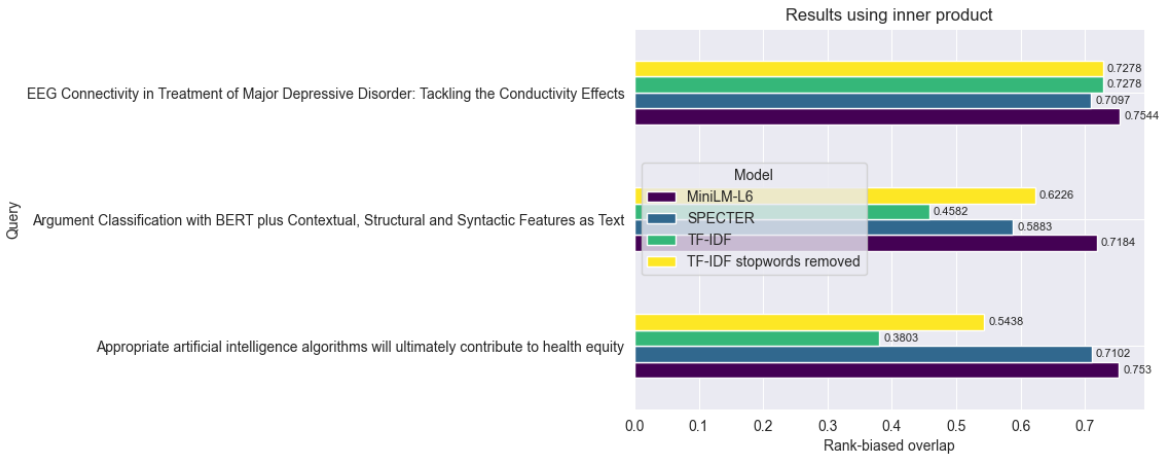

Figure 5.2: Results using dot product, source: author

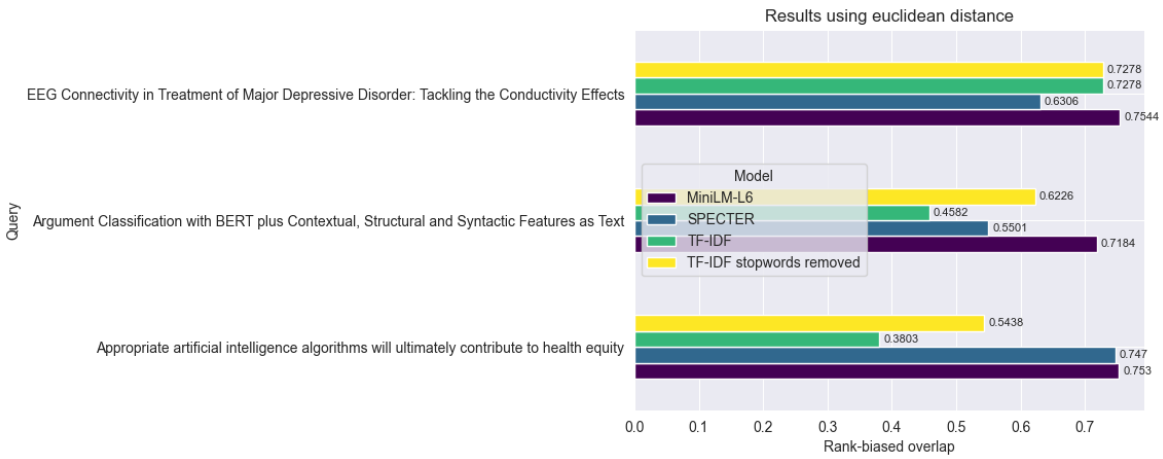

Figure 5.3: Results using Euclidean distance, source: author

According to these experiments, the choice of a text representation when using a vector database matters. From this thesis's set of tested options, *MiniLM-L6* is the safest bet for encoding texts to consistently get reliable results from querying a vector database.

6. Comparison of vector databases

The comparison of used database systems, Chromadb and Qdrant, was done using the author's personal experience with learning, creating collections, querying the databases, trying out different settings and working with small and large datasets - Articles, Aerospace and Projects with 318, 543 and 19,338 rows respectively.

Both implementations are obtained as Python libraries (*chromadb* (Huber, 2024) and *qdrant-client* (Vasnetsov, 2024)) which allow working with an API and a local database client.

Another difference is in the metrics that are used to compute the (dis)similarity of documents. This is explained more in 3.1. In the set of possible measures, ChromaDB allows *squared L2*, *inner product* and *cosine similarity* (Qdrant, 2024b). Qdrant, on the other hand, lets its users choose from *Euclidean distance*, *dot product*, *cosine similarity* and *Manhattan distance* (Qdrant, 2024c). During the initialisation of a collection, a similarity measure is necessary in both applications. Qdrant also requires the user to set the vector size.

When working with the Projects dataset, ChromaDB had problems inserting all the documents simultaneously and required a workaround that utilised batches. Without batching, users are unable to enter large collections of data at once. On the other hand, Qdrant had no problem with the size of the dataset as the points were inserted.

The durations of inserting the data and querying one small and one longer text were chosen to measure the respective databases' performance. All collections' creations and querying took less than 1 second, so there are not too many visible and measurable differences. In case of aerospace dataset insertion, the MiniLM embedding function (default for Chroma) took around 10 seconds on Chroma and around 15 seconds on Qdrant. This difference grew when testing with the Projects dataset to around 367 on Chroma and around 800 seconds on Qdrant. This could be caused by Chroma's optimisation of its default embedding function. Trying out the SPECTER model (provided to both databases as a custom embedding function) has shown that insertion of Aerospace texts took Chroma 30 seconds and Qdrant close to 43 seconds. The duration of insertion of Projects documents on Chroma was circa 26 minutes and on Qdrant more than 30 minutes. The tables of respective observed durations can be found in the thesis attachments *chroma_durations.csv* and qdrant_durations.csv. These experiments were run on an Intel Core i5-13500 CPU.

The one noticeable difference was in the folder structure of the local database clients. When having multiple collections, Qdrant keeps all collections in a *"collection"* folder which contains the set of collections as separate folders with names as specified by the user as collection names and with *storage.sqlite* file inside. This allows the user, for example, to manually delete collections without using the API. Chroma keeps its *chroma.sqlite* file on the user-specified path and its collections in folders named with a hash string that is hard to tie to any existing collection.

Both are very easy to use and have starting tutorials available as part of the documentation; however, Qdrant seems more customisable, and I found its documentation more convenient and easy to read, which is only my personal preference.

	ChromaDB	**Qdrant**
Upsert time (small dataset, custom embedding function)	10 s	15 s
Upsert time (large dataset, custom embedding function)	cca 26 min	cca 32 min
Query time	< 1 s	< 1 s
Allows sparse vectors	no	yes
Upserting data type	list	dict
Allows local client	yes	yes
Ease of use	beginner friendly	beginner friendly
Folder structure	hash as name	collection name as folder name
Allows custom embeddings	yes	yes
Distance metrics	squared L2 inner product cosine similarity	Euclidean distance dot product cosine similarity Manhattan distance

Table 6.1: Comparison of used databases, source: author's rendition

7. Discussion

In this thesis, we experimented with two vector databases, ChromaDB and Qdrant and compared them with respect to their speed, ease of use and customisability. The results of this comparison cannot be generalised, and the *"best database"* cannot be chosen as they are both useful. The assessment of their ease of use was done only by the author's observation without an industry-specific use case.

The experiment conducted with the Articles dataset has shown differences in the order of retrieved results based on the chosen text representation. In this experiment, the TF-IDF collection's performance may be undervalued as no stemming or lemmatisation was applied during preprocessing, no hyperparameter tuning took place, and only default hyperparameters were used. A difference in performance between collections with and without stopwords was observable. The SPECTER model was also not used to its best ability, as it is made to represent title-abstract pairs, and in this thesis, it was used only on abstracts for the sake of consistency with other representations, as the title can bear additional information.

As the experiments were limited by the used hardware and the specificity of the datasets, future improvements can be in utilizing larger and more generic document sets with more ranked documents to compare to the query results.

The metrics of choice for this work were precision at k and rank-biased overlap due to the low cutoff in human-sorted results; however for different use cases, F1 score or Kendall's tau could bear more meaning.

Conclusion

This work introduced basic vector database usage and some differences from the user perspective of two open-source vector database implementations, Chromadb and Qdrant, were shown. In chapter 2, we briefly dived into some selected text representations: Term Frequency-Inverse Document Frequency, pre-trained sentence embeddings of Minilm-L6 and document embeddings of SPECTRE and some standard distance measures used for similarity search: cosine similarity, dot product, Euclidean distance and Manhattan distance. We expected that the choice of representation of the document and the distance metric would have an impact on the ranking of documents retrieved by similarity search. The experiments using these parameters have shown that there is a difference when using certain representations; however, changing the distance metric displayed slight variance only when working with the SPECTER function. This experiment was done on approximately 300 documents concerning scientific papers and journals from the field of computer science. In the following research, more embedding functions, along with more customisable options, can be explored, such as reindexing and HNSW index configuration. If these vector databases were used on a web page or as a part of some application, the next step on top of similarity search would be creating a recommendation system or RAG, a retrieval augmented generation system.

The following dataset, containing around 500 rows of data about aerospace companies, was used to test some of the capabilities of both database implementations. This data was accompanied by a large dataset containing around 19 thousand rows about research and development projects. The goal was to compare the user experience of the two vector databases. In the chapter 5, some differences from the user's perspective are explained; however, these observations are my own, and every user can have different preferences. The experiments have shown that Chroma is faster than Qdrant with the insertion of the documents. On the other hand, ChromaDB is less customisable.

References

AI, E. (2025). Spacy: Industrial-strength natural language processing in python [Accessed: 2025-05-11]. https://pypi.org/project/spacy/

Cohan, A., Feldman, S., Beltagy, I., Downey, D., & Weld, D. S. (2020). SPECTER: Document-level Representation Learning using Citation-informed Transformers. *ACL*.

Huber, J. (2024). Chromadb: Open-source embedding database [Accessed: 2025-05-11]. https://pypi.org/project/chromadb/0.5.13/

Chen, C. (2023). Rbo: Python package for rank-biased overlap [Accessed: 2025-05-11]. https://pypi.org/project/rbo/

Chroma. (2024). Chroma documentation [Accessed: 2024-11-30]. https://docs.trychroma.com/

Kotu, V., & Deshpande, B. (2019). Chapter 4 - classification. In V. Kotu & B. Deshpande (Eds.), *Data science (second edition)* (Second Edition, pp. 65–163). Morgan Kaufmann. https://doi.org/https://doi.org/10.1016/B978-0-12-814761-0.00004-6

MSV, J. (2023). Exploring chroma: The open source vector database for llms [Accessed: 2025-05-11]. https://thenewstack.io/exploring-chroma-the-open-source-vector-database-for-llms/

Nakatani, S. (2010). Language detection library for java. https://github.com/shuyo/language-detection

Pan, J. J., Wang, J., & Li, G. (2023). Survey of vector database management systems. https://arxiv.org/abs/2310.14021

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Pilehvar, M. T., & Camacho-Collados, J. (2021). *Embeddings in natural language processing: Theory and advances in vector representations of meaning* (1st ed.). Springer Cham. https://doi.org/10.1007/978-3-031-02177-0

Qdrant. (2024a). Qdrant-client [Accessed: 2024-11-30]. https://github.com/qdrant/qdrant-client

Qdrant. (2024b). Qdrant-client [Accessed: 2024-11-30]. https://docs.trychroma.com/docs

Qdrant. (2024c). Qdrant-client [Accessed: 2024-11-30]. https://qdrant.tech/documentation

Qdrant. (2025). Qdrant customers [Accessed: 2025-05-11]. https://qdrant.tech/customers/

Reimers, N., & Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks [Accessed: 2025-05-11]. https://arxiv.org/abs/1908.10084

scikit-learn developers. (2025). Text feature extraction [Accessed: 2025-05-11]. https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

The Business Research Company. (2025). Vector database global market report 2025 [Accessed: 2025-05-11]. https://www.thebusinessresearchcompany.com/report/vector-database-global-market-report

Vasnetsov, A. (2024). Qdrant-client: Python client for qdrant vector search engine [Accessed: 2025-05-11]. https://pypi.org/project/qdrant-client/1.12.1/

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. https://arxiv.org/abs/2002.10957

Webber, W., Moffat, A., & Zobel, J. (2010-11). A similarity measure for indefinite rankings. *ACM Trans. Inf. Syst.*, *28*(4). https://doi.org/10.1145/1852102.1852106

Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., Zhong, S., Yin, B., & Hu, X. (2024). Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data*, *18*(6), 1–32.